A Systematization and Empirical Evaluation of Browser-Origin Isolation for Author-Supplied Educational Content — Moodle, WordPress, Omeka S, SCORM, H5P, and eXeLearning

Ernesto Serrano Collado · info@ernesto.es

2026-06-14

Índice

Moodle, WordPress, Omeka S, SCORM, H5P, and eXeLearning

Ernesto Serrano Collado · Independent Researcher, Spain · ORCID: 0009-0006-3817-1317 · info@ernesto.es

Personal capacity. Conflict-of-interest disclosure: the author collaborates on the eXeLearning project and is author/maintainer of several evaluated pieces (`mod_exelearning`, `wp-exelearning`, and `omeka-s-exelearning`); the secure mode described in Section 6.2 is the author's own contribution. See the provenance table in Section 9.

Abstract

Interactive educational resources —SCORM, H5P, eXeLearning packages, HTML pages— often need JavaScript to work. The problem is not JavaScript: it is executing untrusted JavaScript inside an authenticated LMS session without an explicit isolation boundary. This is a **systematisation (SoK) and empirical security evaluation** of eight common ways of publishing content in Moodle, WordPress, and Omeka S (in their stable releases), plus the three maintained integrations with the **proposed** secure mode (implemented as draft PRs), conducted with the source code in hand and an innocuous capability probe run in a local lab. The central finding, organised in a comparative matrix under a single mental model (does it run author JS? same origin as the platform? is there real isolation?), is that when content runs in the same origin as the platform it can read the authenticated DOM, token-bearing forms, and use the session; when isolated (opaque origin, `sandbox` without `allow-same-origin`), it cannot.

We distinguish three states precisely: (a) the *analysed stable releases* of the eXeLearning integrations and Moodle's native SCORM run content in the same origin; (b) for the *maintained eXeLearning integrations*

(`mod_exelearning`, `wp-exelearning`, `omeka-s-exelearning`) we **propose** an opaque-origin secure mode —implemented as draft PRs and validated as a prototype— that closes this exposure; (c) `mod_page`, native SCORM, and `mod_exeweb`/`mod_exescorm` remain *same-origin* by design. H5P is a separate case: it does not execute author HTML, only curated libraries. On `mod_page` we correct a point the literature often confuses: its real protection is not server-side sanitisation (it uses `noclean=true`) but capability/role restriction (`mod/page:addinstance`).

Generative AI acts as a frequency factor —it makes it trivial to paste unreviewed HTML/JS—; we do not measure it empirically, we use it as motivation and threat context. Effective mitigation lives on the server and in the headers, not in trusting that content "will not find" the token. We present and browser-verify (in Chromium and Firefox 146/Gecko) a hardening pattern —opaque origin + validated `postMessage` bridge + CSP— that preserves interactivity and tracking without treating content as part of the platform. Throughout, we separate the evaluated external state (stable releases, own and third-party), the own-contribution mitigation (the secure mode, Section 6.2; provenance table in Section 9), and the proposed future work (Section 6.3).

> **Thesis:** the primary risk of interactive educational resources is not JavaScript, but executing **author** JavaScript within the authenticated same origin of the LMS. A model based on opaque origin, strict `sandbox`, CSP, and a validated `postMessage` bridge keeps interactivity without trusting content as if it were part of the platform.

Keywords: web security; LMS; Moodle; eXeLearning; SCORM; H5P; WordPress; Omeka S; iframe isolation; same-origin policy; `sandbox`; Content Security Policy; `postMessage`; XSS; AI-generated content.

---

1. Introduction

Today it is trivial to ask an AI assistant to "make me an interactive activity in HTML" and paste the result into an LMS resource. That HTML may carry `<script>`, `<iframe>`, `onerror=…`, `fetch()`, or third-party libraries that nobody has reviewed —a transitive-trust risk measured at web scale [1] and aggravated by outdated libraries with known vulnerabilities [2]. At the same time, forum templates, StackOverflow snippets, social-media embeds, and analytics trackers are copied and pasted verbatim. A package uploaded by an author —internal or external— that is rendered inside the session of a teacher or an administrator inherits, absent isolation, part of the privileges of that session. The literature already flags content and resources as a risk vector in e-learning systems [3] and in online learning applications generally [4]; recent empirical analyses of LMS vulnerabilities (Moodle, Chamilo, ILIAS) corroborate this [5]. In the Open Educational Resources space, CEDEC/INTEF warns that pasting AI-generated code one cannot explain "loses the O in Open" and may hide unsafe functionality [6].

The key distinction is between legitimate interactivity and untrusted code: a physics simulation or a quiz needs JS; the risk appears when that JS comes from a source the LMS treats as trusted without having verified it.

We frame this work as a **systematisation (SoK) and empirical security evaluation** —a *design study*—, not a vulnerability report: the value is in the systematic comparison of mechanisms and in the mitigation pattern, not in isolated findings (the nature of the problem is bounded below).

Research question. To what extent do the common ways of publishing interactive educational resources in Moodle, WordPress, and Omeka S isolate author JavaScript from the platform's authenticated session?

Sub-questions:

- **RQ1.** Which integrations execute author JavaScript?
- **RQ2.** Which execute it in the same origin as the platform?
- **RQ3.** Which mitigations preserve interactivity and tracking without exposing the authenticated DOM?
- **RQ4.** How does generative AI change these resources' *operational threat model*? (a plausibility and frequency factor, **not measured** empirically.)

Contributions. (1) A **systematisation** of the risk of interactive educational content along three axes —does it run author JS? same origin as the platform? is there real isolation?— with an explicit classification criterion (Section 3.4); (2) an empirical evaluation of eight publishing mechanisms (in their stable releases, plus three maintained mitigated variants) across Moodle, WordPress, and Omeka S, anchored to `file:line` and verified in the lab, with a *claim → evidence* table (Section 4.1); (3) a set of innocuous, reproducible PoCs (booleans only, no reusable payloads); (4) a mitigation pattern —opaque origin + strict `sandbox` + CSP + validated `postMessage`

bridge— compatible with interactivity and SCORM tracking, split into design requirements (Section 6.2.1) and a reference implementation (Section 6.2.2); and (5) a discussion —not a measurement— of the effect of generative AI and Open Educational Resources (OER).

Nature of the finding (not a 0-day). It is worth situating the type of problem. We distinguish four categories: **(a)** *vulnerability* —a concrete, fixable flaw (e.g. an evadable XSS)—; **(b)** *risk by design* —expected, documented behaviour that is dangerous if content is untrusted (SCORM's same-origin, `noclean=true` in `mod_page`)—; **(c)** *misconfiguration* —overly broad capabilities or roles—; and **(d)** *supply chain* —H5P libraries, third-party JS, or AI-generated content. This article does **not** claim third-party 0-days: it evaluates **trust boundaries** in the publishing of educational resources and, in particular, the **absence of an origin boundary** between author content and the authenticated LMS session.

2. Background

Same-Origin Policy (SOP). The browser isolates documents by *origin* (scheme + host + port). A script may read the DOM, cookies, or storage only of documents in its own origin; cross-origin access throws `SecurityError`. The SOP is the boundary that decides whether a resource's content can "see" the LMS session.

`iframe sandbox`. The `sandbox` attribute restricts what an iframe can do; the tokens (`allow-scripts`, `allow-same-origin`, `allow-popups`, `allow-forms`, `allow-top-navigation`…) re-enable specific capabilities. Critical point: combining `allow-scripts` with `allow-same-origin` over content from the same origin **nullifies the isolation** —the browser itself warns that the content can "escape"— [7], [8]. Without `allow-same-origin`, the document obtains an **opaque origin** (`null`) and stays isolated from the parent. Moreover, **`sandbox` flags propagate to nested iframes**: a YouTube player embedded inside an opaque iframe inherits the restriction and loses its own origin.

Content Security Policy (CSP). A header that limits which origins the document may load scripts, images, or frames from, or connect to (`script-src`, `img-src`, `frame-src`, `connect-src`, `frame-ancestors`, `object-src`, `base-uri`, `sandbox`…) [9]. CSP whitelists are fragile; nonce/`strict-dynamic`-based strategies are recommended [10], and longitudinal analyses show that **deploying an effective CSP in production is hard** [11].

SCORM, H5P, eXeLearning. SCORM specifies that content (the *SCO*) communicate with the LMS through a JavaScript object (`window.API` in 1.2, `API_1484_11` in 2004) discovered by walking `window.parent` [12], [13], [14]. H5P renders *content types* (libraries) curated by the administration, not author HTML [15]. eXeLearning exports packages with the author's HTML/JS (`.elpx`) that the integrations embed in an iframe.

3. Methodology

3.1 Platforms and versions

We analysed: `mod_exelearning`, `mod_exeweb`, `mod_exescorm`, native SCORM (`mod_scorm`), H5P (`mod_h5pactivity` / `core_h5p`), the *Page* resource (`mod_page`), `wp-exelearning`, and `omeka-s-exelearning`. The analysed stable releases (those that fix the "current state" of each platform) are: **Moodle 5.0.7** (core `2104c372962`) with `mod_exelearning` `2c5473d`, `mod_exeweb` `60d24fb`, `mod_exescorm` `e985f4d`, and the eXeLearning editor `8101f54e`; **WordPress** (via `wp-env`) with `wp-exelearning`; and **Omeka S** (Docker, image `erseco/alpine-omeka-s:develop`) with `omeka-s-exelearning`. The full per-`file:line` matrix is in `matriz-seguridad.md`; the per-platform and per-browser appendices are in `anexos-tecnicos.md`.

> **State note.** The **opaque-origin secure mode** described in Section 6.2 is a **proposed mitigation** (implemented as draft PRs, validated as a prototype; upstream adoption pending) for the maintained integrations (`mod_exelearning`, `wp-exelearning`, `omeka-s-exelearning`), distinct from the *same-origin* behaviour of the stable releases analysed here as the "current state". The article explicitly separates the two.

3.1.1 Corpus selection criteria

We selected the mechanisms by three criteria: **(i) prevalence** —common ways of publishing author content in Moodle, WordPress, and Omeka S—; **(ii) execution or *embedding* of author HTML/JS** (we excluded mechanisms that do not run author code, such as text-only resources or links); and **(iii) coverage of the**

trust spectrum: from *no isolation* (top window: `mod_page`; unsandboxed iframe: native SCORM, `mod_exeweb`, `mod_exescorm`), through *semantic filtering* (H5P), to *configurable opaque-origin isolation* (`mod_exelearning`, `wp-exelearning`, `omeka-s-exelearning`). The set is **representative** of the three content origin relationships that determine the risk (top *same-origin*, iframed *same-origin*, opaque iframe), not an exhaustive sample of every existing plugin; resources that do not run author JS and platforms outside the study are therefore out of scope.

3.2 Environment

Local, disposable Docker instances; lab course/page marked `POC-SAFE`. The live browser tests were run with **two engines**: a Chromium-based one and **Firefox (Gecko 146)** via Playwright. We verified live `mod_exelearning`, `mod_scorm`, `mod_h5pactivity`, `mod_page`, `wp-exelearning`, and `omeka-s-exelearning`; the rest (`mod_exeweb`, `mod_exescorm`) were verified against source code.

Cross-browser verification. To confirm that opaque-origin isolation is **behaviour defined by the web standard** and not that of one particular engine, we replicated the check in **Firefox 146 (Gecko)** with a reproducible Playwright script (`evidencias/firefox-isolation-test.cjs`). **(i)** In a self-contained test, an iframe with `sandbox` *with* `allow-same-origin` reads `window.parent` (inherited origin), whereas *without* `allow-same-origin` the access throws `SecurityError` and `isOpaqueOrigin` is `true` —measured **from inside** the iframe itself—. **(ii)** The real secure-mode embeds of `mod_exelearning` (served via `tokenpluginfile`), `wp-exelearning`, and `omeka-s-exelearning` are **opaque** in Firefox too: `contentDocument === null` and `contentWindow` throws `SecurityError` in all **three** integrations. The result is **identical to Chromium's** (`evidencias/resultados-firefox.json`, `evidencias/resultados-firefox-moodle.json`).

3.3 The probe (`probe.js`): definition of each measure

The proofs of concept share a probe that runs a series of checks and returns **only** booleans and *redacted* error names. It never reads real values, never makes network calls, never `POST`s, never invokes SCORM mutators. Each measure:

Boolean	What it detects (never exercises)
`canRunJavascript`	the browser runs author script
`isOpaqueOrigin`	the document runs in an opaque origin (`null`)
`sandboxAttr` / `sandboxAllowsSameOrigin`	effective `sandbox` attribute and whether it grants `allow-same-origin`
`canAccessParent` / `canReadParentDocument`	whether `window.parent/parent.document` are reachable (same origin)
`canReadParentCookie`	whether `parent.document.cookie` is readable (value always `REDACTED`)
`canFindSesskey`	whether the `sesskey`/nonce is present in the same-origin DOM (value `REDACTED`)
`canFindCourseEditForms` / `canFindCourseEditLinks`	presence of edit forms/links
`canAccessTop` / `canAttemptTopNavigation`	top-window reachability (does not navigate; `not_attempted`)
`canOpenPopups`	opens and immediately closes a 1×1 popup (harmless)
`canUsePostMessage` / `canPostMessageToParent`	channel availability (sends nothing)
`canCallScormApi` / `scormApiFlavor`	whether `window.API/API_1484_11` is reachable (does not invoke it)
`canUseLocalStorage` / `canUseSessionStorage`	same-origin storage access
`sandboxEscape`	**always** `false`: detected by design, never attempted

Four packages encapsulate the probe: `evil.elpx` (eXeLearning), `evil.h5p` (negative control: H5P filters it), `evil-scorm.zip` (SCORM 1.2 that **detects** `window.API`), and `evil-page.html` (inline probe for *Page*). Typical *redacted* output:

```
{ "canRunJavascript": true, "canAccessParent": false, "canReadParentDocument": false,
  "canReadParentCookie": false, "parentCookieValue": "REDACTED", "canFindSesskey": false,
  "sesskeyValue": "REDACTED", "canCallScormApi": true, "isOpaqueOrigin": true,
  "sandboxEscape": false, "error": "SecurityError" }
```

3.4 Risk-classification criteria

We classify each mechanism with an explicit criterion over **seven** observable dimensions: (i) does it run author JS?; (ii) in the same origin as the platform?; (iii) the minimum role to *publish* the content; (iv) the role of the *viewer* who runs it; (v) is there a session token readable in the same-origin DOM?; (vi) is a restrictive CSP emitted?; (vii) is there a reachable server-side mutating capability? From these we separate the *maximum demonstrated impact* (verified in the lab) from the *inferred impact* (deduced from the browser model). The three-level lay summary:

- **Low:** does not run author JS, or runs it in an isolated origin (opaque; e.g. `sandbox` without `allow-same-origin`) with no parent access. (`srcdoc`/subdomain are equivalent architectural alternatives, not evaluated here.)
- **Medium:** runs isolated JS but with residual channels (popups, `postMessage`), or filtered by evadable semantics (documented XSS).
- **High:** runs author JS in the same origin as the LMS (access to the authenticated DOM/session), or in the top window without a sandbox.

The **formal risk matrix** per platform —these seven dimensions plus demonstrated/inferred impact— is in `matriz-seguridad.md` (Section 1).

3.5 Ethical limits of the method

We do not steal cookies or tokens, we do not exfiltrate anything, and we do not attack external systems. **The distributed probe (`probe.js`) makes no network calls and no `POST`**: it only **detects** capabilities and prints a table of *redacted* booleans. To **confirm impact** (Section 4.2 and appendices) we did execute, in an **authorised and reversible** way, real requests —including `POST`— **only on the author's own or lab accounts**, never destructive, never in production, and never against third parties; the changes (e.g. the author's own profile name/photo) were reverted. We do not publish reusable payloads. Detail in Section 8 (Ethics and responsible disclosure).

4. Results

4.1 Main table

Platform / resource	Runs author JS?	Same origin as LMS?	Real isolation	Risk level
`mod_page` (Page)	Yes (`noclean=true`; verified)	Yes (top window, no iframe)	None server-side; only gated by `mod/page:addinstance`	High if a teacher edits it (runs in every viewer's session)
`mod_scorm` (core)	Yes	Yes	None (no sandbox)	High
`mod_h5pactivity` / `core_h5p`	Params: No (filtered). Libraries: Yes (`preloadedJs`, trusted code)	Yes	Params filtered by semantics; library JS runs *same-origin*, unsandboxed	Low for content; high if libraries can be installed (`h5p:updatelibraries`, manager/admin)
`mod_exelearning` (stable)	Yes	Yes	Partial (`sandbox` with `allow-same-origin`)	Medium-high
`mod_exelearning` (secure mode)	Yes	No (opaque)	Strong (opaque origin + bridge)	Low
`mod_exeweb`	Yes	Yes	None	High
`mod_exescorm`	Yes	Yes	None	High
`wp-exelearning` (stable)	Yes	Yes	Partial (`sandbox` with `allow-same-origin`)	Medium-high
`wp-exelearning` (secure mode)	Yes	No (opaque)	Strong (opaque origin)	Low
`omeka-s-exelearning` (stable)	Yes	Yes	Partial (`sandbox` with `allow-same-origin`)	Medium-high
`omeka-s-exelearning` (secure mode)	Yes	No (opaque)	Strong (opaque origin)	Low

For each maintained integration we show **two states**: the evaluated **stable** release (*same-origin*) and the **proposed secure mode** (opaque origin; draft PRs). The `legacy` (*same-origin*) mode remains an **optional** fallback —e.g. for third-party embeds that need their own origin, or when the benefit is judged to outweigh the risk—; the *same-origin* exposure is detailed in Section 4.5.

Quick reading (answers RQ1–RQ2): executing author JavaScript is not the problem; the problem is executing it in the same origin as the LMS and without an explicit boundary.

4.1.1 Claim → evidence

So the reader can tell, without ambiguity, what is verified live, what in code, and what is pending:

Claim	Type of evidence	Source
`mod_page` executes author `<script>`	Live (Moodle 5.0.7 local) + code	4.2; `mod/page/view.php:90-93`

Claim	Type of evidence	Source
Native SCORM runs *same-origin* without a sandbox	Live + code	4.3; `player.php:279-285`
H5P filters the *parameters* (they do not execute)	Live (negative control)	4.4; `poc/evil.h5p`
A library's `preloadedJs` runs *same-origin*	Code + structurally-valid PoC + manual procedure	4.4; `resultados-h5p-library.json`
The secure mode isolates (opaque origin)	Live in Chromium and Firefox 146/Gecko	4.5–4.6; `resultados-firefox*.json`
`mod_exeweb` / `mod_exescorm` *same-origin*, unsandboxed	Code only (inference)	matrix 2.2
Persistent self-edit of one's own profile (`legacy`)	Live, authorised and reversible (own account)	appendix (live confirmation)
Safari / WebKit	Not verified (future work)	—

4.2 `mod_page` (Page) — protection is the capability, not sanitisation

A user with the capability to create a Page writes HTML. When displayed, `mod_page` calls `format_text(..., noclean=true)` (`mod/page/view.php:90-93`, `lib.php:352`). We created a Page with `<script>` and `<img onerror>` and opened it: **both executed**. With `noclean=true` (and `forceclean=0` by default), `format_text` does **not** go through `purify_html()`; the author's `<script>` is stored and executed for **whoever views it (including students)**, in the **main window** and in the **same origin** as Moodle.

The real protection is therefore **not server-side filtering** —there is none— **but capability/role**: only authorised roles can create/edit a Page (`mod/page:addinstance`). The `$CFG->enabletrusttext` flag is off by default, but it does **not** condition execution in `mod_page`, because this resource uses `noclean`. We thus correct a common formulation: the defence is not "Moodle filters `<script>`" but "only a teacher or manager can publish the Page".

Impact on a student's session (verified in code). The script —same-origin, top window— can: (a) not read the session cookie (`MoodleSession` is `HttpOnly` by default), but it can *use the session* by reading `M.cfg.sesskey` and, since Moodle's main page does not emit a restrictive CSP, exfiltrate the `sesskey` and DOM data and forge authenticated requests; (b) modify the view's DOM and persistently change the user's own profile (name/photo by replaying the `user/edit.php` form, confirmed on a real Moodle); (c) send messages on their behalf —`core_message_send_instant_messages` is `'ajax' => true` with `moodle/site:sendmessage`, a capability of the `user` role—, enabling *interaction-dependent propagation* (the recipient must open a link; the message body is sanitised on display, so it does not auto-execute).

Self-propagation limit. A student **cannot** plant executable HTML: forums use `trusttext` (with `enabletrusttext` off they are sanitised) and the student lacks `addinstance`. Propagation **escalates** if the person who opens the Page is a teacher or manager: with their privileges the script can create more Pages and call privileged services (e.g. `core_user_update_users`, `'ajax' => true`).

Scope: confined to the origin, but a latent vector. By the SOP, the script cannot read the cookies/DOM of the user's other websites (banking, email), nor saved passwords, nor other tabs; the scope is limited to the LMS session itself. Even so, having arbitrary JS in the authenticated origin is a persistent foothold: it could exploit a future vulnerability reachable from that origin (a service without a capability check, an IDOR/CSRF), and its impact **is bounded by the capabilities of the role that opens it** (if an administration profile opens it, it executes administrative actions). It also need not act immediately: the script can stay dormant —harmless to students— and condition its payload on who opens the page (`M.cfg.userId`/roles, DOM elements visible only to management profiles, or capability probing), so it activates only for a privileged role. It is, in the worst case, a targeted and patient attack, always **subject to the viewer's capabilities**.

The same messaging channel (`core_message_send_instant_messages`) could also *induce* a higher-privilege profile to open the resource, subject to messaging policy: by default (`messagingallusers=0`) only to contacts/coursemates, and to anyone only if `messagingallusers` is enabled. The impact, once opened, is **bounded by that profile's capabilities**: with course creation or management it could create a course (we reproduced this by scraping `course/edit.php`) and enrol learners (`enrol_manual_enrol_users`, in the courses it manages); an administration profile could modify accounts or site configuration. That is, escalation depends on role and configuration; it is not automatic.

The opaque origin removes that foothold; `mod_page` does not have it.

4.3 Native SCORM (`mod_scorm`)

The SCO walks `window.parent`/`window.opener` looking for `window.API` (`mod/scorm/loadSCO.php`). The iframe is created **without a `sandbox` attribute** (`player.php`) and the content is served from the same origin (`pluginfile.php`). SCORM, by design, assumes same origin and parent access; isolating it would make it incompatible. Its defence is **server-side**: `confirm_sesskey()` + the `mod/scorm:savetrack` capability before saving tracking. Risk: a malicious SCORM executes JS with Moodle's origin (reads the DOM and rides the session); the validation limits *which server actions* it can force, not the reading of context. It is **the least client-isolated** of those analysed. Additional standard limitation: since tracking happens on the client, students can falsify `completion`/`score` with `LMSSetValue`, documented for years [16]; the integrity of digital assessment is a field in its own right [17].

4.4 H5P (`mod_h5pactivity` / `core_h5p`)

H5P injects the content into an `about:blank` iframe via `contentDocument.write()`; that document **inherits the parent origin** (it is not opaque, **with no** `sandbox` attribute: `h5piframe.mustache:32-34`), so its isolation does **not** come from the origin. What it controls is **what** it executes, and here two planes must be distinguished.

Parameter plane (negative control). The authoring text of `content.json` passes through `H5PContentValidator`: `validateText` applies `filter_xss()` with a **closed tag allowlist** that **does not include `<script>`** and `_filter_xss_attributes` drops the `on*` attributes (`h5p.classes.php:4303-4384`, `:5033-5054`); without `tags`, the field is escaped with `htmlspecialchars`. We verified this: `evil.h5p` injects `<script>`/`<img onerror>` into a text field and H5P discards them. A `.h5p` that relies on the **parameters** to execute JS is, therefore, a **negative control**. Even so, allowlist-based sanitisation is **historically fragile** —`innerHTML` mutations (mXSS) and client-side XSS evade filters that do not parse the DOM [18], [19]— so robustness should not rest on the filter alone.

Library plane (protection is the capability, not sanitisation). But H5P **libraries are trusted code by design**: their `preloadedJs` loads as `<script src=pluginfile.php/…/core_h5p/…>` and runs **same-origin and unsandboxed** in the Moodle page (`player.php:484-500`, `h5p.js:391-437`). The H5P documentation states this without ambiguity —*"JavaScript files … are by default and necessity allowed for H5P libraries but not for H5P content"*— and recommends that *"only trusted users should be given permission to update h5p libraries"* [20]. The only barrier to author content introducing its **own** library is a **capability**, not a filter: installing a new library from an uploaded `.h5p` requires `moodle/h5p:updatelibraries` (archetype **manager**, `RISK_XSS`), evaluated **against the uploader** (`api.php:403-405`, `helper.php:210-224`, `h5p.classes.php:1577-1579`). Teachers hold only `moodle/h5p:deploy` (they can use already-installed libraries, not install new ones); a package requiring an absent library is **rejected at validation**. We built `evil-h5p-library.h5p` (the library `H5P.ExePocAlert`, whose `preloadedJs` shows a notice and read-only capability booleans). That its `preloadedJs` runs **same-origin and unsandboxed** is **verified against source code** (the `file:line` paths cited) and the package is **structurally valid**; live end-to-end execution is documented as a **reproducible manual procedure** (upload with a management role → the notice appears when viewing the content), since the **headless automation** of Moodle 5's file picker proved unreliable and remains as future automation work. It is exactly the `mod_page` pattern: the real defence is **capability/role**, not sanitisation or a sandbox —and the risk is one of **admin-trust / supply-chain**. This vector is documented in the real world: in Chamilo, importing a malicious `.h5p` reached **authenticated RCE** (`CVE-2026-30875`) [21].

H5P is not immune on the content side either: a history of evadable XSS —`MDL-67110` (JS execution) [22], `CVE-2024-43439` (reflected XSS via an H5P error message, which bypasses the content filter) [23], `CVE-2024-3111` (stored XSS via SVG upload escalating to a backdoor in the WordPress H5P plugin) [24]—; and its `postMessage` validates `event.source` and `context==='h5p'` but **not** `event.origin`, sending with `'*'` —the kind of check [25] found exploitable on 84 popular sites.

> **Nuanced verdict:** H5P does not execute the HTML/JS of the *parameters* (it filters them: negative control), but the libraries are trusted code running *same-origin*, unsandboxed; what separates author content from executing JavaScript is the `moodle/h5p:updatelibraries` capability (manager/admin), not sanitisation. Same pattern as `mod_page`. Evidence: `evidencias/resultados-h5p-library.json`; PoC: `poc/evil-h5p-library.h5p`.

4.5 eXeLearning in Moodle — same-origin baseline and secure mode (`mod_exelearning`, `mod_exeweb`, `mod_exescorm`)

In the stable releases, `.elpx` is extracted and served by `pluginfile.php` and shown in an iframe with `sandbox="allow-scripts allow-same-origin allow-popups allow-forms allow-popups-to-escape-sandbox"`. It is the **best-isolated of the three eXe integrations in Moodle** (it blocks `allow-top-navigation` and `allow-modals`), but it keeps `allow-same-origin` for three dependencies of the synchronous SCORM bridge (the parent reads `iframe.contentDocument` to map `objectid`; *pipwerks* walks `window.parent`; the *teacher-mode hider* injects CSS into the `contentDocument`). With both flags over content from the same origin, the `sandbox` does **not really isolate**. `mod_exeweb` shows the content **without `sandbox`** and `mod_exescorm` does not sandbox either: both are **weaker**. We verified live (in `legacy` mode) that the iframe content reads `parent.M.cfg.sesskey` and forges `core_user_update_users` (it renamed a lab account, reverted).

4.6 eXeLearning in WordPress and Omeka S

In the stable releases, `wp-exelearning` embeds the package with `sandbox="allow-scripts allow-same-origin allow-popups"` → **same origin**; the probe obtained `canAccessParent: true`, `canReadParentDocument: true` and located links to `/wp-admin/`. In addition, its REST proxy `/content/<hash>` has `permission_callback => '__return_true'` (unauthenticated read of the content by hash). In `omeka-s-exelearning`, the item's public view used `sandbox="allow-same-origin allow-scripts allow-popups allow-popups-to-escape-sandbox"`; the probe measured `canAccessParent: true`, `canReadParentCookie: true`, `isOpaqueOrigin: false` → **same origin** (Omeka does impose mandatory CSRF validation). WordPress and Omeka have no `sesskey`; they use nonces and CSRF tokens, but the logic is the same: the risk is not that the content "sees" the token, but that the server accepts an action because it comes with a valid token that a same-origin script can read from the DOM.

5. Discussion

Implications for teachers. Pasted-from-AI or copied JS runs, in most stable integrations, with the LMS origin and in the session of **every** viewer. Legitimate interactivity does not require entrusting the origin to the content.

Implications for administrators. The critical surface is *who* can publish HTML/JS (`mod/page:addinstance`, `moodle/site:trustcontent`) and *with what isolation*. Moodle does not emit a global CSP by default; it is advisable to adopt an opaque-origin mode (such as the one proposed in Section 6.2) and to treat external packages as untrusted. It is worth remembering that the *absence of symptoms does not prove safety*: a malicious same-origin script can stay dormant and activate only when an administrator opens it (a targeted, patient attack, Section 4.2); the prevalence of persistent client-side XSS is empirically documented [26]. XSS in Moodle is the subject of continuous auditing [27], [28].

Impact of generative AI (RQ4) — motivation, not measurement. We do not measure generative AI empirically; we treat it as a factor that changes the *operational threat model*: it does not introduce a new vulnerability, but it makes the dangerous pattern more frequent —plausible HTML/JS content, copied without review, published by a trusted role—, turning a latent risk into an everyday one. This is a plausibility-and-frequency hypothesis, not a quantitative result; measuring it (e.g. with a usage study) remains future work. CEDEC/INTEF frames it for OER: a resource with unreviewed AI code may be legally open but pedagogically and technically closed and insecure [6].

Compatibility versus security. The secure mode's central dilemma: the opaque origin isolates, but it **breaks third-party embeds** that need their own origin (YouTube/Vimeo), because the `sandbox` propagates to the nested iframe. Resolving this without giving up isolation is the subject of Sections 6.2–6.3.

6. Mitigations

We separate the external state analysed (6.1) from the own-contribution mitigation (6.2 —split into *design requirements*, 6.2.1, and a *reference implementation* in eXeLearning, 6.2.2—) and from the proposed mitigations (6.3).

6.1 External state: the author-trust model

Moodle, at its core, **trusts the author** for embeds: the media filter recognises known providers by URL allow-list (the `core_media_player_external` classes for YouTube/Vimeo) and embeds the iframe **directly, without `sandbox`**; H5P trusts its curated libraries; SCORM trusts the uploaded package. It is a reasonable model when authorship is trusted (teachers), but it does not isolate untrusted content.

6.2 Mitigation: design requirements and a reference implementation

6.2.1 Design requirements (implementation-independent) To serve author HTML/JS without exposing the session, an implementation should meet, regardless of the platform:

1. **Opaque origin by default.** `sandbox` with `allow-scripts` (and `allow-popups`/`allow-forms` as needed) **without `allow-same-origin`**; the isolated mode as the default and any *same-origin* mode only as a fallback, with *fail-safe* normalisation to the secure mode.
2. **A single source of truth** for the `sandbox` tokens (one helper), instead of strings duplicated per template.
3. **A `sandbox` directive in the CSP of the response** (not only in the iframe attribute): the document retains the opaque origin even if opened outside the iframe (new tab, popup, raw URL).
4. **Restrictive CSP:** `connect-src 'self'` (cuts off exfiltration), `frame-ancestors 'self'` / `X-Frame-Options: SAMEORIGIN` (anti-clickjacking), `object-src 'none'`, `base-uri 'none'`, a bounded `form-action`, and a `Permissions-Policy` that disables camera/microphone/geolocation/payment.
5. **Neutralisation of the package's active types** (SVG/XML) with a script-free CSP.
6. **No direct parent iframe access.** Where crossing is needed (teacher mode, SCORM grading), resolve it server-side (state via the `src`) or via validated `postMessage` —window identity + nonce + a closed action list— with the credentials (`sesskey`/nonce) only in the parent and server-side re-validation.
7. **Serving by a read-only capability:** a token that only reads files (not the `sesskey`) or a content proxy with an explicit `Content-Type` and path-traversal protection.
8. **Security tests:** that the expected `sandbox` is present per mode, that the message handler rejects unknown origins/sources, and that the default mode is the isolated one.

6.2.2 Reference implementation (draft PRs) in the eXeLearning integrations We **propose** a secure mode that instantiates R1–R8 for the three maintained integrations (`mod_exelearning`, `wp-exelearning`, `omeka-s-exelearning`), **implemented as draft PRs** —the author's own contribution; upstream adoption is pending; see the provenance table in Section 9—. We **validated the prototypes** in the browser across all three (opaque origin confirmed in Chromium and Firefox 146/Gecko; benign content still renders). Before/after measurement: in `legacy` the content reads `parent.M.cfg.sesskey` and forges an action; in `secure` the same attempt throws `SecurityError` (opaque origin). In Moodle, R7 is met with `tokenpluginfile` (a token that only reads files) and the SCORM bridge (R6) transmits only the score via validated `postMessage`, with the `sesskey` exclusively in the parent.

Threat table (asset · threat/condition · impact · mitigation):

Asset	Threat (condition)	Impact	Mitigation
LMS session	same-origin JS uses the session (runs author JS in the LMS origin)	authenticated actions (per role)	opaque origin (no `allow-same-origin`)
Authenticated DOM	parent read from the iframe (with `allow-same-origin`)	data/nonce exposure	`sandbox` without same-origin + a directive in the response
SCORM tracking	client-side score tampering (JS API reachable same-origin)	falsified grades	validated `postMessage` bridge + server-side re-validation
File token	exfiltration of the read-only token (the CSP admits `https:`)	temporary access to package files	strict-CSP profile (proposed, 6.3) + short TTL
Privileged user	dormant, targeted payload (waits for or induces a manager/administrator to open it)	bounded by that profile's capabilities (course creation, enrolment…)	the opaque origin removes the foothold; review by role
Other viewers	propagation via message (AJAX messaging, role `user`)	interaction-dependent	content confined to the origin; review by role

Assumed limitation. The opaque origin is incompatible with third-party embeds that need their own origin (YouTube/Vimeo): the `sandbox` propagates to the nested player. To avoid degrading the experience, the secure mode renders the video via a *parent-mediated overlay* (the content requests promoting an iframe; the parent,

outside the sandbox, validates and overlays the real player). Current implementation: a provider allowlist with canonical-URL reconstruction. Generalisation to any provider is treated in Section 6.3.

6.3 Proposed mitigations (future work)

- **Video from any provider without a whitelist (structural invariant).** Instead of keeping a host allowlist (YouTube/Vimeo) with per-provider reconstruction, promote any iframe whose `src` is **https + cross-origin to the LMS** (rejecting same-origin/subdomain/IP/loopback/userinfo). The security argument: a **cross-origin** iframe cannot read the LMS (the SOP protects the parent), exactly the trust model Moodle already uses to embed YouTube; the "cross-origin" invariant replaces the host list and admits YouTube, Vimeo, Dailymotion, Mediateca de Madrid, and any provider **without enumerating them** or creating subdomains. Trade-off to document: the author could embed any cross-origin content (a phishing/tracking risk, not an escape); for high-security deployments, a "strict mode" option can re-enable a list. A more conservative alternative: **server-side oEmbed** (the LMS asks the provider for the embed HTML), safer but limited to providers with oEmbed and with a server-side fetch cost [29].
- **Optional strict-CSP profile.** Close the detected residual: a script in the opaque iframe can still exfiltrate the file token via `<img src="https://attacker/?t=TOKEN">` because `img-src`/`script-src`/`media-src` admit `https:`. An optional profile (admin, off by default so as not to break external images/MathJax/CDN) that limits those directives to the package's assets closes the channel.
- **Complementary platform defences.** Secure-by-default mechanisms enforced by the browser, such as **Trusted Types**, have eliminated DOM-XSS at scale in large code bases [30]; they are complementary to opaque-origin isolation —they restrict the dangerous *capability* at the platform boundary, the same logic we apply to `mod_page` and the H5P libraries.
- **Configurability** of the embed helper by the administration (parity across the three integrations).

7. Limitations

A local and disposable environment (we did not measure production); specific versions (the results are tied to the commits in Section 3.1); no destructive exploitation (we demonstrate the chain, not the abuse); verified in **two engines** (Chromium and Firefox 146/Gecko), with **Safari/WebKit not tested** (future work); the H5P library vector is verified in code and with a structurally-valid PoC, with end-to-end execution as a manual procedure (the headless automation of Moodle 5's file picker was unreliable); `mod_exeweb`/`mod_exescorm` are inferred from code, not a live test; generative AI (RQ4) is not measured. The threat model focuses on client-side isolation, not the entire LMS surface. Generalisation to other versions or configurations requires re-verification against the corresponding code.

8. Ethics and responsible disclosure

The behaviours described are, for the most part, documented and by design: `mod_page` with `noclean=true`, SCORM's same-origin, and the trust model of the media filter and the H5P libraries are not third-party 0-days but known design decisions gated by capability. The secure mode of Section 6.2 is a mitigation **proposed and implemented as draft PRs** in the author's own software (the author's own contribution; upstream adoption pending).

Responsible disclosure. No coordinated third-party disclosure was required: (i) the behaviours evaluated are documented/by-design, not reportable defects, so they were not reported as vulnerabilities; (ii) the only link to a third-party flaw is the citation of `CVE-2026-30875` (Chamilo), which was already public and patched *upstream* before this work; (iii) no unpatched third-party 0-day was found. Should such a finding arise in the future, it would be coordinated with the maintainers before disclosure. We publish *redacted* PoCs (booleans + errors only), with no reusable payloads and no abuse steps, and the reversible lab changes were reverted.

9. Conflict-of-interest statement

The author is a collaborator of the eXeLearning project and author/maintainer of several pieces of the analysed ecosystem. This dual role —analyst and developer of part of the object of study— is disclosed for transparency; it does not invalidate the results (verifiable against the code and artifacts), but the reader should be aware of it.

To separate it unambiguously, the following **provenance table** distinguishes what is evaluated, the author's tie, and the evidence:

Component	Author's tie	Role in the study	Evidence
Moodle core (`mod_page`, `mod_scorm`)	None (third party)	Evaluated object	Live + code
H5P (`core_h5p`)	None (third party)	Evaluated object	Parameters: live · library: code + structurally-valid PoC + manual
SCORM (standard)	None (third party)	Evaluated object	Live + code
`mod_exeweb`, `mod_exescorm`	eXeLearning ecosystem (no declared tie)	Evaluated object	Code only (inference)
`mod_exelearning`, `wp-exelearning`, `omeka-s-exelearning`	Author/maintainer	Evaluated object (stable) **and** proposed mitigation (secure mode, 6.2.2; draft PRs)	`legacy`: live · `secure`: live on prototype (Chromium + Firefox 146)
Safari / WebKit	—	Browser coverage	Pending (future work)

Claims about the author's own software rest on the same citable evidence (`file:line`, evidence JSON, PoC) as those about third parties; no own mitigation is evaluated by the author's inspection alone.

10. Artifact availability

A reproducible artifact bundle is published at **https://github.com/erseco/lms-untrusted-content-security-paper** (paper text under **CC-BY-4.0**; code and PoCs under **MIT**): the `probe.js` probe (15 redacted checks), the PoC packages and their `build.sh` —including the H5P library `evil-h5p-library.h5p` that supports reproducing `preloadedJs` execution—, the full per-`file:line` **matrix** (`matriz-seguridad.md`), the per-platform/per-browser **appendices** (`anexos-tecnicos.md`), the JSON evidence (`evidencias/`, including the Firefox cross-browser verification of the three integrations with their Playwright scripts, and `resultados-h5p-library.json`), and the document-generation script, together with a reproducibility guide (`REPRODUCIBILITY.md`), a `Makefile` with the build targets, and the checksums of the published PDFs (`pdf/SHA256SUMS`). The analysed versions are identified by the commits in Section 3.1. The PoCs contain no reusable payloads; the copyrighted source PDFs are not redistributed (they are linked by DOI/URL).

11. Conclusions

JavaScript is not the enemy: interactive educational content often needs it. The risk arises from executing untrusted JavaScript in the same origin as the LMS (RQ1–RQ2): of what was analysed, H5P is the most controlled by design (it does not execute author HTML), `mod_page` is protected by capability/role (not by sanitisation), and stable SCORM/eXeLearning run *same-origin* for compatibility. The answer to RQ3 is a concrete, verified pattern — opaque origin + strict `sandbox` + CSP (incl. a response-level directive) + a validated `postMessage` bridge— which keeps interactivity and tracking without exposing the authenticated DOM; we propose and implement that pattern (as draft PRs) for the maintained eXeLearning integrations. On RQ4, generative AI does not create the flaw and we do not measure it: we pose it as a frequency factor that makes the pattern routine. We keep separate the evaluated external state, the own-contribution mitigation (Sections 6.2 and 9), and the future work (Section 6.3), so the reader can distinguish the evaluation from the author's own patch. The rule that sums it up: **isolate, validate, and do not trust the resource's JavaScript as if it were part of the platform.**

Technical appendices (full per-`file:line` matrix, redacted PoCs, per-platform/per-browser results, methodological limitations): see `anexos-tecnicos.md` and `matriz-seguridad.md`.

12. Generative AI use statement

Generative AI tools (LLM-based writing and coding assistants) were used in preparing this work to support drafting and restructuring the text, generating and refactoring the proof-of-concept and evidence scripts, and producing tables. **AI is not listed as an author.** The author designed the research, defined the methodology, ran and verified every test in the lab environment, manually checked each technical claim, the code, and the cited evidence, and takes **full responsibility** for the content.

13. References

[1] N. Nikiforakis *et al.*, "You are what you include: Large-scale evaluation of remote JavaScript inclusions," in *Proc. 2012 ACM conference on computer and communications security (CCS '12)*, 2012, pp. 736–747. doi: 10.1145/2382196.2382274.

[2] T. Lauinger, A. Chaabane, S. Arshad, W. Robertson, C. Wilson, and E. Kirda, "Thou shalt not depend on me: Analysing the use of outdated JavaScript libraries on the web," in *Proc. 2017 network and distributed system security symposium (NDSS '17)*, 2017. doi: 10.14722/ndss.2017.23414.

[3] A. Khamparia and B. Pandey, "Threat driven modeling framework using petri nets for e-learning system," *SpringerPlus*, vol. 5, no. 1, p. 446, 2016, doi: 10.1186/s40064-016-2101-0.

[4] A. J. J. Joseph and M. Mariappan, "Approaches to overcome security risks and threats in online learning applications," in *Secure data management for online learning applications*, CRC Press, 2023. doi: 10.1201/9781003264538-3.

[5] S. A.-L. Akacha and A. I. Awad, "Enhancing security and sustainability of e-learning software systems: A comprehensive vulnerability analysis and recommendations for stakeholders," *Sustainability*, vol. 15, no. 19, p. 14132, 2023, doi: 10.3390/su151914132.

[6] CEDEC (Centro Nacional de Desarrollo Curricular en Sistemas no Propietarios), "Mantener la «a» de abierto en los REA en tiempos de IA." Accessed: June 14, 2026. [Online]. Available: https://cedec.intef.es/mantener-la-a-de-abierto-en-los-rea-en-tiempos-de-ia/

[7] MDN Web Docs, "The inline frame element: The sandbox attribute." Accessed: June 13, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTML/Element/iframe#sandbox

[8] WHATWG, "HTML living standard — sandboxing." Accessed: June 13, 2026. [Online]. Available: https://html.spec.whatwg.org/multipage/browsers.html#sandboxing

[9] S. Stamm, B. Sterne, and G. Markham, "Reining in the web with content security policy," in *Proceedings of the 19th international conference on world wide web (WWW)*, 2010, pp. 921–930. doi: 10.1145/1772690.1772784.

[10] L. Weichselbaum, M. Spagnuolo, S. Lekies, and A. Janc, "CSP is dead, long live CSP! On the insecurity of whitelists and the future of content security policy," in *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security (CCS)*, 2016, pp. 1376–1387. doi: 10.1145/2976749.2978363.

[11] S. Roth, T. Barron, S. Calzavara, N. Nikiforakis, and B. Stock, "Complex security policy? A longitudinal analysis of deployed content security policies," in *Proc. 2020 network and distributed system security symposium (NDSS '20)*, 2020. doi: 10.14722/ndss.2020.23046.

[12] Advanced Distributed Learning (ADL), "SCORM 1.2 run-time environment," Advanced Distributed Learning Initiative, 2001. Accessed: June 13, 2026. [Online]. Available: https://adlnet.gov/projects/scorm/

[13] Advanced Distributed Learning (ADL), "SCORM 2004 4th edition — run-time environment (API API_1484_11)," Advanced Distributed Learning Initiative, 2009. Accessed: June 13, 2026. [Online]. Available: https://adlnet.gov/projects/scorm/

[14] P. Hutchison, "SCORM API wrapper for JavaScript (pipwerks)." Accessed: June 13, 2026. [Online]. Available: https://github.com/pipwerks/scorm-api-wrapper

[15] H5P Group, "H5P documentation — content types and libraries." Accessed: June 13, 2026. [Online]. Available: https://h5p.org/documentation

[16] P. Hutchison, "Cheating in SCORM." Accessed: June 13, 2026. [Online]. Available: https://pipwerks.com/2009/03/22/cheating-in-scorm/

[17] P. Dawson, *Defending assessment security in a digital world: Preventing e-cheating and supporting academic integrity in higher education*. Routledge, 2020. doi: 10.4324/9780429324178.

[18] M. Heiderich, J. Schwenk, T. Frosch, J. Magazinius, and E. Z. Yang, "mXSS attacks: Attacking well-secured web-applications by using innerHTML mutations," in *Proc. 2013 ACM SIGSAC conference on computer and communications security (CCS '13)*, 2013, pp. 777–788. doi: 10.1145/2508859.2516723.

[19] B. Stock, S. Lekies, T. Mueller, P. Spiegel, and M. Johns, "Precise client-side protection against DOM-based cross-site scripting," in *Proc. 23rd USENIX security symposium (USENIX security '14)*, 2014, pp. 655–670. Available: https://www.usenix.org/system/files/conference/usenixsecurity14/sec14-paper-stock.pdf

[20] H5P, "Security (H5P documentation): Libraries are trusted code." Accessed: June 14, 2026. [Online]. Available: https://h5p.org/documentation/installation/security

[21] MITRE CVE, "CVE-2026-30875: Authenticated remote code execution in chamilo LMS via H5P package import." Accessed: June 14, 2026. [Online]. Available: https://github.com/chamilo/chamilo-lms/security/advisories/GHSA-mj4f-8fw2-hrfm

[22] Moodle Tracker, "MDL-67110: XSS in malicious seemingly H5P content." Accessed: June 13, 2026. [Online]. Available: https://tracker.moodle.org/browse/MDL-67110

[23] Deutsche Telekom Security and Moodle, "CVE-2024-43439: Reflected XSS in Moodle via H5P error message." Accessed: June 13, 2026. [Online]. Available: https://github.security.telekom.com/2024/08/reflected-xss-moodle.html

[24] CleanTalk PSC, "CVE-2024-3111: Interactive content – H5P (WordPress) stored XSS to backdoor creation." Accessed: June 13, 2026. [Online]. Available: https://research.cleantalk.org/cve-2024-3111/

[25] S. Son and V. Shmatikov, "The postman always rings twice: Attacking and defending postMessage in HTML5 websites," in *Proceedings of the network and distributed system security symposium (NDSS)*, 2013. Available: https://www.ndss-symposium.org/ndss2013/ndss-2013-programme/postman-always-rings-twice-attacking-and-defending-postmessage-html5-websites/

[26] M. Steffens, C. Rossow, M. Johns, and B. Stock, "Don't trust the locals: Investigating the prevalence of persistent client-side cross-site scripting in the wild," in *Proc. 2019 network and distributed system security symposium (NDSS '19)*, 2019. doi: 10.14722/ndss.2019.23009.

[27] R. R. Al-azaiza and T. S. M. Barhoom, "Enhance MOODLE security against XSS vulnerabilities," *International Journal of Computing and Digital Systems*, vol. 5, no. 5, pp. 421–430, 2016, doi: 10.12785/ijcds/050507.

[28] Sonar, "Playing dominos with Moodle's security." Accessed: June 13, 2026. [Online]. Available: https://www.sonarsource.com/blog/playing-dominos-with-moodles-security-2/

[29] oEmbed, "oEmbed — an open format for embedding content via a provider API." https://oembed.com/.

[30] P. Wang, B. Á. Gumundsson, and K. Kotowicz, "Adopting trusted types in production web frameworks to prevent DOM-based cross-site scripting: A case study," in *2021 IEEE european symposium on security and privacy workshops (EuroS&PW)*, 2021, pp. 60–73. doi: 10.1109/EuroSPW54576.2021.00013.